

BitPruning: Learning Bitlengths for Aggressive and Accurate Quantization

Miloš Nikolić
University of Toronto
Toronto, Canada

Ghouthi Boukli Hacene
Mila
Montreal, Canada

Ciaran Bannon
University of Toronto
Toronto, Canada

Alberto Delmas Lascorz
University of Toronto
Toronto, Canada

Matthieu Courbariaux
Mila
Montreal, Canada

Yoshua Bengio
Mila
Montreal, Canada

Vincent Gripon
IMT-Atlantique
Brest, France

Andreas Moshovos
University of Toronto
Toronto, Canada

Abstract—Neural networks have demonstrably achieved state-of-the-art accuracy using low-bitlength integer quantization, yielding both execution time and energy benefits on existing hardware designs that support short bitlengths. However, the question of finding the minimum bitlength for a desired accuracy remains open. We introduce a training method for minimizing inference bitlength at any granularity while maintaining accuracy. Namely, we propose a regularizer that penalizes large bitlength representations throughout the architecture and show how it can be modified to minimize other quantifiable criteria, such as number of operations or memory footprint. We demonstrate that our method learns thrifty representations while maintaining accuracy. With ImageNet, the method produces an average per layer bitlength of 4.13, 3.76 and 4.36 bits on AlexNet, ResNet18 and MobileNet V2 respectively, remaining within 2.0%, 0.5% and 0.5% of the base TOP-1 accuracy.

Index Terms—Heterogeneous quantization, learned datatype, neural networks

I. INTRODUCTION

Over the past two decades energy and power (energy over time) have emerged as the primary constraints for computing devices [1] dictating execution time performance, operating costs, or up-time for virtually all device segments, from data centers to the edge. Accordingly, reducing Deep Neural Network (DNN) energy needs can yield a multitude of benefits such as improved latency for existing models, enable the deployment of more powerful models, or boost throughput and reduce operating costs for data centers. A key choice that dictates DNN energy consumption and thus execution time during inference is the choice of *datatype*, that is the number of bits used per value (activations or weights) and their numerical interpretation (e.g., floating-point or fixed-point). For example, using 8b fixed-point vs. 32b floating-point reduces the energy of a multiply-accumulate by more than $23\times$ [2], whereas using 4b fixed-point more than halves energy compared to 8b and doubles compute bandwidth even on existing hardware [3]. Choosing a datatype may at first appear a simple task. However, doing so naively can leave a

lot of the potential benefits untapped, or worse can yield a network that fails to converge.

While 32-bit floating point is definitely sufficient, many DNNs can use other lower cost floating- or fixed-point representations without sacrificing accuracy. For this reason, and while earlier graphics processors (GPUs) and general-purpose processors (CPUs) supported only a few datatypes, newer generations of GPUs and CPUs have been extended to support additional more energy efficient datatypes specifically targeting DNNs [3]–[5]. Support for multiple datatypes is a key feature of specialized hardware accelerators [6]–[9].

Energy efficiency improves when using a narrower datatype (e.g., 16b instead of 32b floating-point) or a simpler and less expensive to implement one (e.g., fixed-point vs. floating-point). More energy efficient datatypes yield benefits in two ways (see Table I for example accelerators):

Memory: Using narrower datatypes reduces the number of bits that need to be stored and transferred. This greatly reduces energy as off-chip memory accesses are today one order of magnitude slower and two orders of magnitude more energy consuming compared to on-chip operations (accesses or computation). Doing so enables us to run larger and more powerful networks, to deploy devices with smaller on- and off-chip memories, and to enjoy higher performance especially when memory bandwidth is limited, as it is often the case today. Besides being able to directly benefit CPUs and GPUs today, this observation also motivated several low-cost memory compression techniques that directly take advantage of narrower datatypes at layer [10] [7] [8] or finer granularity [11] [12].

Computation: The operations per cycle throughput of many hardware platforms scales inversely proportionally with datatype bit length. Most commodity general purpose or graphics processors (CPUs or GPUs) support several bit lengths, e.g., 16b at 1x and 8b at 2x throughput, whereas, spatially composable and bit-serial accelerators [7], [8], [13]–[17] can support a full range of bit lengths. Moreover, using simpler datatypes such as fixed-point vs. floating-point can also amplify compute bandwidth. For example, many commodity CPUs and GPUs implement more fixed-point units than floating-point

Correspondence to: Miloš Nikolić <milos.nikolic@mail.utoronto.ca>.